CSV Processor

Solution & Architecture Document

1. Project Overview

The CSV Processor is a containerized Python/Flask web application designed to handle stock-on-hand (SOH) CSV exports. It provides a browser-based upload interface, parses each row, and persists processed files and history to AWS S3, enabling a consistent experience across all replicas.

Core Capabilities

- Accepts CSV file uploads in SOH headerless format or generic CSV with headers.
- Parses rows and displays parsed results in the browser.
- Uploads processed files to S3 under a timestamped key path.
- Maintains cluster-wide upload history via S3, consistent across all pod replicas.
- Falls back to local JSON storage when S3 is unavailable for minikube and development environments.

Platform Stack

Component	**Technology / Detail**
Application runtime	Python 3 / Flask + gunicorn (2 workers per pod)
Static file serving	nginx sidecar container per pod
Container registry	Docker Hub — barbhua786/csv-processor
Orchestration	Kubernetes (kops self-managed or AWS EKS)
Infrastructure-as-Code	kops cluster spec + Terraform (EKS alternative)
Deployment automation	Helm chart + Ansible for environment-specific configuration
Object storage	AWS S3 (processed/ and history/ prefixes)
Autoscaling	HPA (CPU-based) + Cluster Autoscaler (node-level)
Credential management	IRSA — IAM Roles for Service Accounts (no secrets)

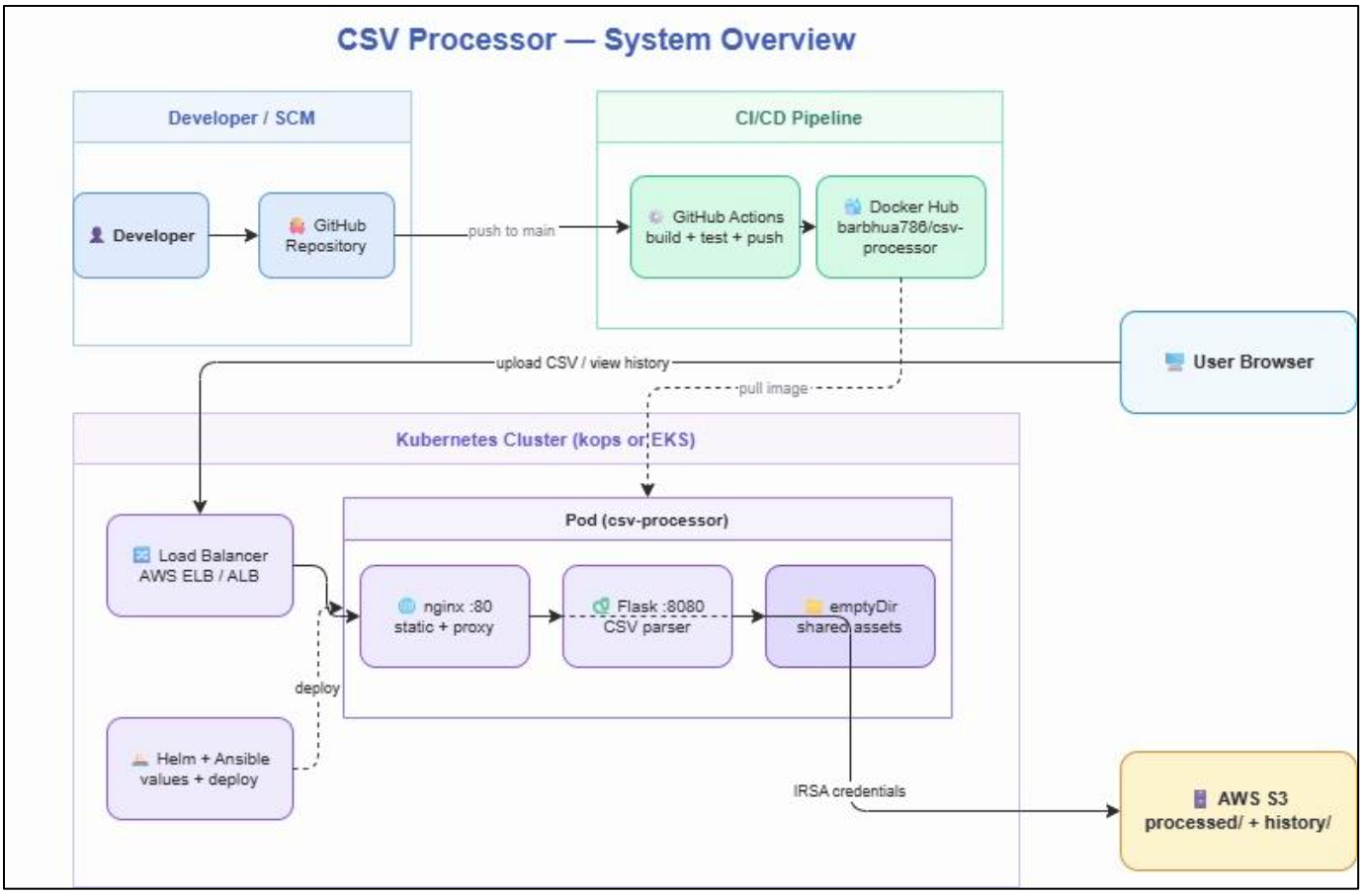
2. Application Design

The Flask application supports both SOH headerless CSV input and generic CSV with headers. It auto-detects the format, strips BOM characters, skips blank lines, and returns parsed output as a list of line-and-field dictionaries. Processed CSV files are archived to S3 under processed/{stamp}/{filename}, while upload history is written to history/{stamp}.json using microsecond-precision timestamps to avoid collisions under concurrent load.

3. Architecture

2.1 System Overview

The system spans three tiers: a CI/CD pipeline, a Kubernetes runtime, and AWS S3 persistent storage.



2.2 Repository Structure

The solution is organized as a single monorepo in the main repository, https://github.com/asif-ahmedb/csv-processor, with the application, infrastructure, and Kubernetes deployment assets maintained as top-level folders inside it rather than as independent repositories. This preserves clear separation of concerns while keeping all related components in one source-controlled location, which simplifies coordinated changes, shared documentation, and automation across the full delivery workflow.

Folder / Area	Contents / Purpose	Key Details
csv-processor-app	Application source code and container build assets.	Contains the Flask web application, templates, static assets, pytest suite, nginx configuration for non-Kubernetes mode, Docker entrypoint assets, sample data, and the Dockerfile.
csv-processor-infrastructure	AWS cluster provisioning assets for cloud deployment.	Provides both kops-based self-managed Kubernetes assets and a Terraform-based EKS alternative, including IAM templates, manifests, scripts, and environment examples.
csv-processor-k8s-assets	Kubernetes deployment assets for all environments.	Includes the Helm chart, environment-specific values files, Ansible automation, rendered values, Minikube deployment scripts, and deployment quick-reference guides.

2.3 Kubernetes Pod Design

Each pod runs two containers sharing an emptyDir volume for static assets.

Container	Image	Port	Role
initContainer	barbhua786/csv-processor	—	Copies CSS/JS assets to shared emptyDir at startup
nginx	nginx:alpine (official)	:80	Serves static assets and reverse-proxies /api/* to Flask
Flask app	barbhua786/csv-processor	:8080	Parses CSV and writes files and history entries to S3

2.4 CSV Upload Sequence

1. Browser sends POST /api/process with a multipart CSV file to nginx.
2. nginx reverse-proxies the request to Flask on localhost:8080.
3. Flask parses rows, detects SOH or generic format, and strips BOM if present.
4. Flask uploads the processed file to S3 using the key pattern processed/{timestamp}/{filename}.csv.
5. Flask writes a history record to S3 using the key pattern history/{timestamp}.json.
6. Flask returns JSON containing rows and history entry details; the browser renders the result.
7. The browser can retrieve upload history through /api/history, which lists and fetches items under the history/ prefix.

3. Infrastructure Design

3.1 Minikube — Local Development Option

Minikube provides a lightweight local Kubernetes environment for development and testing. It is well suited for validating the application, Helm chart behavior, and service exposure without requiring AWS infrastructure. While useful for local workflows and demonstrations, it does not model the production-grade AWS identity, persistence, or autoscaling features used in cloud deployments.

Aspect	**Minikube (Local)**
Target use case	Developer testing, demos, and local validation
Cluster topology	Single-node local Kubernetes cluster
Service exposure	NodePort or kubectl port-forward
Storage behavior	Local persistence or emptyDir depending on values
S3 dependency	Optional; can be disabled or simulated locally
Credential model	No IRSA required for local-only execution
Best suited for	Fast iteration before deploying to AWS-based environments

3.2 kops — Self-Managed Kubernetes (Primary)

kops is the primary deployment approach in AWS for this solution, offering full control over cluster topology, node groups, and autoscaling behavior. It supports a mix of on-demand and spot worker groups to balance baseline reliability with cost efficiency. The design

assumes a small control plane with multiple worker pools and uses supporting automation to configure S3 access, IAM roles, and the cluster-autoscaler.

Group Name	Role	Type	Min/Max	Purpose
master-us-east-1a	Control plane	t3.small	1 / 1	Private subnet; runs etcd and API server
nodes-ondemand	Worker	t3.small	1 / 3	Guaranteed capacity; always-on baseline
nodes-spot	Worker	t3.small/t3.medium	0 / 5	Pure spot; zero cost when idle
nodes-mixed	Worker	t3.small/t3.medium	1 / 4	On-demand base + spot; balanced cost and reliability

3.3 Terraform / EKS — Managed Alternative

The Terraform-based EKS option provides a managed alternative to kops for teams that prefer AWS to operate the Kubernetes control plane. It provisions the network, EKS cluster, node groups, S3 bucket, IAM policy, and IRSA role using infrastructure-as-code modules. This option reduces cluster management overhead while preserving the same application deployment model and S3-based persistence pattern.

Terraform Resource	Module / Type	Purpose
VPC + subnets + NAT	terraform-aws-modules/vpc/aws ~5.0	Network foundation with private and public subnets
EKS cluster + nodes	terraform-aws-modules/eks/aws ~20.0	Managed control plane and worker node groups
S3 bucket	aws_s3_bucket + lifecycle rules	Encrypted, versioned object storage
IAM policy for S3	aws_iam_policy	Least-privilege PutObject/GetObject/ListBucket access
IRSA role	terraform-aws-modules/.../iam-role-for-service-accounts-eks	Credential-less pod access to S3

4. Kubernetes & Autoscaling Design

The Helm chart supports multiple environments through values files: base defaults, local minikube, kops on AWS, and EKS/Terraform. CPU-based horizontal pod autoscaling is used because CSV parsing is CPU-bound, with production configurations typically starting at two replicas and scaling upward based on CPU utilization. Cluster Autoscaler adds

nodes when pods are unschedulable and prefers mixed-capacity groups before spot or on-demand fallback.

5. S3 Storage & Lifecycle

The solution stores uploaded CSV archives under a processed/ prefix and metadata history under a history/ prefix inside an account-specific S3 bucket. Lifecycle rules transition processed objects to Glacier after 30 days, Deep Archive after 180 days, and expire them after 365 days. History JSON files are intentionally kept small and excluded from lifecycle transitions. Security controls include blocked public access, server-side encryption using SSE-S3, bucket owner enforcement, versioning, and IRSA-only access.

6. Security Design

The architecture avoids storing AWS credentials in Kubernetes Secrets or environment variables by using IRSA for temporary, least-privilege access. Containers run as a non-root user (UID 33), privilege escalation is disabled, Linux capabilities are dropped, and writable mounted volumes are aligned with the application user. Networking is restricted through private subnets for masters and workers, with only the necessary load balancers exposed publicly and production CIDRs expected to be limited to office or VPN ranges.

7. Environment Matrix

Feature	**minikube (dev)**	**kops (AWS)**	**EKS (AWS)**
Values file	values-minikube.yaml	values-kops.yaml	values-eks.yaml
Service type	NodePort :30080	LoadBalancer	ClusterIP + Ingress
Persistence	Enabled (dev)	Disabled (emptyDir)	Disabled (emptyDir)
S3	Optional (MinIO/localstack)	Required	Required
IRSA	Not needed	kops pod identity webhook	EKS native OIDC
Autoscaling	min 1 / max 4	min 2 / max 10	min 2 / max 20
Node groups	Single	ondemand + spot + mixed	ondemand + spot
Image registry	Local (minikube image load)	Docker Hub	ECR (prod inventory)